Lesson 6: Advanced RAG Techniques

Take your RAG system to the next level with these advanced techniques used in production systems.

Overview of Advanced Techniques

Technique	Purpose	Complexity	Impact
Semantic Chunking	Better context boundaries	Medium	High
Hybrid Search	Combine semantic + keyword	Medium	High
Re-ranking	Improve result relevance	Low	Medium
Query Expansion	Handle ambiguous queries	Medium	Medium
Document Metadata	Better filtering	Low	Medium
Multi-query Retrieval	Comprehensive coverage	Medium	High
Self-querying	Extract filters from query	High	High
Parent-child Chunking	Context preservation	High	High

1. Advanced Chunking Strategies

Recursive Character Splitting

Split by multiple delimiters in order of preference:

```
from typing import List

class RecursiveChunker:
   def __init__(self, chunk_size: int = 1000, overlap: int = 200):
       self.chunk_size = chunk_size
       self.overlap = overlap
       # Priority order: paragraphs > sentences > words > characters
       self.separators = ["\n\n", "\n", ". ", " ", ""]

   def split_text(self, text: str) -> List[str]:
       """Recursively split text by separators"""
       return self._split_text_recursive(text, self.separators)

   def _split_text_recursive(self, text: str, separators: List[str]) -> List[str]:
       """Recursive splitting logic"""
       if not separators:
           # Base case: split by character
           return self._simple_split(text, "")

       separator = separators[0]
       if separator == "":
           # Character-level split
           return self._simple_split(text, separator)

       # Try to split by current separator
       splits = text.split(separator)

       chunks = []
       current_chunk = []
       current_size = 0

       for split in splits:
           split_size = len(split)

           if current_size + split_size <= self.chunk_size:
               current_chunk.append(split)
               current_size += split_size + len(separator)
           else:
               # Current chunk is full
               if current_chunk:
                   chunks.append(separator.join(current_chunk))

               # If this split is too large, recurse with next separator
               if split_size > self.chunk_size:
                   sub_chunks = self._split_text_recursive(split, separators[1:])
                   chunks.extend(sub_chunks)
                   current_chunk = []
                   current_size = 0
               else:
                   current_chunk = [split]
                   current_size = split_size

       if current_chunk:
           chunks.append(separator.join(current_chunk))

       return self._add_overlap(chunks)
```

```
    def _simple_split(self, text: str, separator: str) -> List[str]:
        """Simple split by character"""
        chunks = []
        for i in range(0, len(text), self.chunk_size - self.overlap):
            chunks.append(text[i:i + self.chunk_size])
        return chunks

    def _add_overlap(self, chunks: List[str]) -> List[str]:
        """Add overlap between chunks"""
        if len(chunks) <= 1:
            return chunks

        overlapped = [chunks[0]]
        for i in range(1, len(chunks)):
            # Add end of previous chunk to start of current
            prev_end = chunks[i-1][-self.overlap:]
            overlapped.append(prev_end + chunks[i])

        return overlapped

# Usage
chunker = RecursiveChunker(chunk_size=500, overlap=50)
chunks = chunker.split_text(long_document)
```

Semantic Chunking

Group sentences by topic similarity:

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
import nltk

class SemanticChunker:
    def __init__(self, threshold: float = 0.7, max_chunk_size: int = 1000):
        self.model = SentenceTransformer('all-MiniLM-L6-v2')
        self.threshold = threshold
        self.max_chunk_size = max_chunk_size
        nltk.download('punkt', quiet=True)

    def chunk_text(self, text: str) -> List[str]:
        """Chunk text based on semantic similarity"""
        # Split into sentences
        sentences = nltk.sent_tokenize(text)

        if len(sentences) == 0:
            return []

        # Get embeddings for all sentences
        embeddings = self.model.encode(sentences)

        # Initialize chunks
```

```
        chunks = []
        current_chunk = [sentences[0]]
        current_size = len(sentences[0])

        for i in range(1, len(sentences)):
            sentence = sentences[i]
            sentence_len = len(sentence)

            # Check similarity with current chunk
            current_embedding = np.mean([embeddings[j] for j in range(i-len(current_chunk), i)
            similarity = cosine_similarity(
                current_embedding.reshape(1, -1),
                embeddings[i].reshape(1, -1)
            )[0][0]

            # Decide whether to add to current chunk or start new one
            if (similarity >= self.threshold and
                current_size + sentence_len <= self.max_chunk_size):
                # Similar topic and not too large - add to current chunk
                current_chunk.append(sentence)
                current_size += sentence_len
            else:
                # Different topic or too large - start new chunk
                chunks.append(' '.join(current_chunk))
                current_chunk = [sentence]
                current_size = sentence_len

        # Add final chunk
        if current_chunk:
            chunks.append(' '.join(current_chunk))

        return chunks

# Usage
chunker = SemanticChunker(threshold=0.75)
chunks = chunker.chunk_text(document)
```

Parent-Child Chunking

Store small chunks but retrieve with larger context:

```
class ParentChildChunker:
    def __init__(self, child_size: int = 200, parent_size: int = 1000):
        self.child_size = child_size
        self.parent_size = parent_size

    def chunk_text(self, text: str, doc_id: str) -> tuple:
        """
        Returns: (child_chunks, parent_chunks, mappings)
        """
        # Create parent chunks
        parent_chunks = []
        start = 0
```

```
        parent_id = 0

        while start < len(text):
            end = min(start + self.parent_size, len(text))
            parent_chunks.append({
                'id': f"{doc_id}_parent_{parent_id}",
                'content': text[start:end],
                'start': start,
                'end': end
            })
            start = end
            parent_id += 1

        # Create child chunks with parent references
        child_chunks = []
        child_id = 0

        for parent in parent_chunks:
            parent_text = parent['content']
            child_start = 0

            while child_start < len(parent_text):
                child_end = min(child_start + self.child_size, len(parent_text))
                child_chunks.append({
                    'id': f"{doc_id}_child_{child_id}",
                    'content': parent_text[child_start:child_end],
                    'parent_id': parent['id']
                })
                child_start = child_end
                child_id += 1

        return child_chunks, parent_chunks

# Usage with RAG
def retrieve_with_parents(query: str, rag_system, n_results: int = 3):
    """Retrieve child chunks but return parent context"""
    # Search child chunks (small, precise)
    child_results = rag_system.retrieve(query, n_results)

    # Get parent chunks for each child
    parent_ids = [r['metadata']['parent_id'] for r in child_results]
    parent_chunks = rag_system.get_chunks_by_ids(parent_ids)

    return parent_chunks  # Return larger context!
```

2. Hybrid Search

Combine semantic search with keyword/BM25 search:

```
from rank_bm25 import BM25Okapi
import numpy as np

class HybridRetriever:
```

```
    def __init__(self, vector_db, documents: List[str], alpha: float = 0.5):
        """
        alpha: weight for semantic search (1-alpha for keyword)
        alpha=1.0: pure semantic, alpha=0.0: pure keyword
        """
        self.vector_db = vector_db
        self.documents = documents
        self.alpha = alpha

        # Initialize BM25
        tokenized_docs = [doc.split() for doc in documents]
        self.bm25 = BM25Okapi(tokenized_docs)

    def search(self, query: str, top_k: int = 5) -> List[dict]:
        """Hybrid search combining semantic and keyword"""
        # 1. Semantic search
        semantic_results = self.vector_db.query(
            query_texts=[query],
            n_results=top_k * 2  # Get more candidates
        )

        # 2. Keyword search (BM25)
        tokenized_query = query.split()
        bm25_scores = self.bm25.get_scores(tokenized_query)

        # 3. Normalize scores to [0, 1]
        semantic_scores = 1 - np.array(semantic_results['distances'][0])
        semantic_scores = (semantic_scores - semantic_scores.min()) / (semantic_scores.max() -

        bm25_scores = (bm25_scores - bm25_scores.min()) / (bm25_scores.max() - bm25_scores.min

        # 4. Combine scores
        combined_scores = {}

        # Add semantic scores
        for i, doc_id in enumerate(semantic_results['ids'][0]):
            combined_scores[doc_id] = self.alpha * semantic_scores[i]

        # Add BM25 scores
        for i, score in enumerate(bm25_scores):
            doc_id = f"doc_{i}"
            if doc_id in combined_scores:
                combined_scores[doc_id] += (1 - self.alpha) * score
            else:
                combined_scores[doc_id] = (1 - self.alpha) * score

        # 5. Sort by combined score
        sorted_docs = sorted(combined_scores.items(), key=lambda x: x[1], reverse=True)

        # 6. Return top-k
        results = []
        for doc_id, score in sorted_docs[:top_k]:
            doc_idx = int(doc_id.split('_')[1])
            results.append({
                'content': self.documents[doc_idx],
                'score': score,
                'doc_id': doc_id
            })

        return results
```

```
# Usage
hybrid = HybridRetriever(vector_db, documents, alpha=0.7)
results = hybrid.search("GPT-4", top_k=5)
```

3. Re-ranking

Re-order retrieved results with a cross-encoder:

```
from sentence_transformers import CrossEncoder

class Reranker:
    def __init__(self, model_name: str = 'cross-encoder/ms-marco-MiniLM-L-6-v2'):
        """Initialize re-ranker with cross-encoder model"""
        self.model = CrossEncoder(model_name)

    def rerank(self, query: str, documents: List[dict], top_k: int = 3) -> List[dict]:
        """
        Re-rank documents using cross-encoder

        documents: List of dicts with 'content' key
        Returns: Top-k documents after re-ranking
        """
        if not documents:
            return []

        # Create pairs of (query, document)
        pairs = [[query, doc['content']] for doc in documents]

        # Get scores from cross-encoder
        scores = self.model.predict(pairs)

        # Add scores to documents
        for doc, score in zip(documents, scores):
            doc['rerank_score'] = float(score)

        # Sort by score
        documents.sort(key=lambda x: x['rerank_score'], reverse=True)

        return documents[:top_k]

# Usage in RAG
def rag_with_reranking(query: str, rag_system, reranker, initial_k: int = 10, final_k: int =
    """RAG with re-ranking step"""
    # 1. Initial retrieval (get more candidates)
    initial_results = rag_system.retrieve(query, n_results=initial_k)

    # 2. Re-rank
    reranked_results = reranker.rerank(query, initial_results, top_k=final_k)

    # 3. Generate answer with top re-ranked results
    answer = rag_system.generate_answer(query, reranked_results)
```

```
    return {
        'answer': answer,
        'sources': reranked_results
    }
```

4. Query Expansion

Generate multiple variations of the query:

```
class QueryExpander:
    def __init__(self, llm):
        self.llm = llm

    def expand_query(self, query: str, n_variations: int = 3) -> List[str]:
        """Generate query variations"""
        prompt = f"""Generate {n_variations} different ways to ask the following question.
Each variation should preserve the original meaning but use different words.

Original question: {query}

Variations (one per line):"""

        response = self.llm.chat.completions.create(
            model="gpt-3.5-turbo",
            messages=[{"role": "user", "content": prompt}],
            temperature=0.7
        )

        # Parse variations
        variations = response.choices[0].message.content.strip().split('\n')
        variations = [v.strip('- ').strip() for v in variations if v.strip()]

        return [query] + variations  # Include original

def retrieve_with_expansion(query: str, rag_system, expander, top_k: int = 5):
    """Retrieve using expanded queries"""
    # Expand query
    queries = expander.expand_query(query, n_variations=2)

    # Retrieve for each variation
    all_results = []
    for q in queries:
        results = rag_system.retrieve(q, n_results=top_k)
        all_results.extend(results)

    # Deduplicate and re-rank by frequency/score
    seen = {}
    for result in all_results:
        doc_id = result['metadata'].get('chunk_id', result['content'][:50])
        if doc_id in seen:
            seen[doc_id]['score'] += 1 - result['distance']
        else:
            result['score'] = 1 - result['distance']
```

```
            seen[doc_id] = result

    # Sort by combined score
    final_results = sorted(seen.values(), key=lambda x: x['score'], reverse=True)
    return final_results[:top_k]
```

5. Multi-Query Retrieval

Break complex queries into sub-queries:

```
class MultiQueryRetriever:
    def __init__(self, llm, rag_system):
        self.llm = llm
        self.rag_system = rag_system

    def decompose_query(self, query: str) -> List[str]:
        """Break complex query into simpler sub-queries"""
        prompt = f"""Break down this complex question into 2-3 simpler sub-questions
that, when answered together, would answer the original question.

Original question: {query}

Sub-questions (one per line):"""

        response = self.llm.chat.completions.create(
            model="gpt-3.5-turbo",
            messages=[{"role": "user", "content": prompt}],
            temperature=0.3
        )

        sub_queries = response.choices[0].message.content.strip().split('\n')
        sub_queries = [q.strip('- ').strip() for q in sub_queries if q.strip()]

        return sub_queries

    def retrieve_multi_query(self, query: str, top_k: int = 3) -> dict:
        """Retrieve using multiple sub-queries"""
        # Decompose query
        sub_queries = self.decompose_query(query)

        # Retrieve for each sub-query
        all_results = {}
        for sub_q in sub_queries:
            results = self.rag_system.retrieve(sub_q, n_results=top_k)
            all_results[sub_q] = results

        # Combine results
        combined = []
        for results in all_results.values():
            combined.extend(results)

        # Deduplicate
        seen_content = set()
```

```
        unique_results = []
        for r in combined:
            if r['content'] not in seen_content:
                seen_content.add(r['content'])
                unique_results.append(r)

        return {
            'sub_queries': sub_queries,
            'results': unique_results[:top_k * 2]  # Return more results for complex queries
        }

# Usage
retriever = MultiQueryRetriever(llm, rag_system)
result = retriever.retrieve_multi_query(
   "How do embeddings work in RAG and what models are commonly used?"
)
```

6. Self-Querying

Extract filters from natural language:

```
class SelfQueryRetriever:
   def __init__(self, llm, rag_system):
        self.llm = llm
        self.rag_system = rag_system

   def parse_query(self, query: str, available_fields: List[str]) -> dict:
        """Extract filters from natural language query"""
        prompt = f"""Extract the search query and any filters from this question.

Available filter fields: {', '.join(available_fields)}

Question: {query}

Return in this format:
Search query: [the core question]
Filters: [field1=value1, field2=value2, ...]

Response:"""

        response = self.llm.chat.completions.create(
            model="gpt-3.5-turbo",
            messages=[{"role": "user", "content": prompt}],
            temperature=0
        )

        # Parse response
        content = response.choices[0].message.content
        lines = content.strip().split('\n')

        search_query = ""
        filters = {}
```

```
        for line in lines:
            if line.startswith("Search query:"):
                search_query = line.split(":", 1)[1].strip()
            elif line.startswith("Filters:"):
                filter_str = line.split(":", 1)[1].strip()
                if filter_str and filter_str != "none":
                    for f in filter_str.split(','):
                        if '=' in f:
                            key, value = f.split('=', 1)
                            filters[key.strip()] = value.strip()

        return {
            'query': search_query or query,
            'filters': filters
        }

    def retrieve(self, query: str, available_fields: List[str], top_k: int = 3) -> List[dict]:
        """Retrieve with automatic filter extraction"""
        # Parse query
        parsed = self.parse_query(query, available_fields)

        # Retrieve with filters
        results = self.rag_system.collection.query(
            query_texts=[parsed['query']],
            n_results=top_k,
            where=parsed['filters'] if parsed['filters'] else None
        )

        return {
            'query': parsed['query'],
            'filters': parsed['filters'],
            'results': results
        }

# Usage
retriever = SelfQueryRetriever(llm, rag_system)
results = retriever.retrieve(
    "What papers about RAG were published in 2023?",
    available_fields=['year', 'topic', 'author']
)
# Extracts: query="papers about RAG", filters={'year': '2023'}
```

7. Contextual Compression

Compress retrieved documents to remove irrelevant parts:

```
class ContextualCompressor:
    def __init__(self, llm):
        self.llm = llm

    def compress_documents(self, query: str, documents: List[str]) -> List[str]:
        """Extract only relevant portions of documents"""
        compressed = []
```

```
        for doc in documents:
            prompt = f"""Extract only the parts of this document that are relevant to the ques
Keep the original wording but remove irrelevant sentences.

Question: {query}

Document:
{doc}

Relevant excerpts:"""

            response = self.llm.chat.completions.create(
                model="gpt-3.5-turbo",
                messages=[{"role": "user", "content": prompt}],
                temperature=0,
                max_tokens=500
            )

            compressed_doc = response.choices[0].message.content.strip()
            if compressed_doc:
                compressed.append(compressed_doc)

        return compressed

# Usage
compressor = ContextualCompressor(llm)
retrieved_docs = rag_system.retrieve(query, n_results=5)
compressed_docs = compressor.compress_documents(
   query,
   [d['content'] for d in retrieved_docs]
)
# Now use compressed_docs for generation - fits more context!
```

8. Production Optimizations

Caching

```
from functools import lru_cache
import hashlib

class CachedRAG:
   def __init__(self, rag_system):
        self.rag_system = rag_system
        self.cache = {}

   def _hash_query(self, query: str) -> str:
        return hashlib.md5(query.encode()).hexdigest()

   def query_with_cache(self, query: str, ttl: int = 3600):
        """Query with caching"""
        query_hash = self._hash_query(query)
```

```
        # Check cache
        if query_hash in self.cache:
            cached_result, timestamp = self.cache[query_hash]
            if time.time() - timestamp < ttl:
                return cached_result

        # Execute query
        result = self.rag_system.query(query)

        # Cache result
        self.cache[query_hash] = (result, time.time())

        return result
```

Async Processing

```
import asyncio
from typing import List

class AsyncRAG:
    def __init__(self, rag_system):
        self.rag_system = rag_system

    async def retrieve_async(self, query: str) -> List[dict]:
        """Async retrieval"""
        loop = asyncio.get_event_loop()
        return await loop.run_in_executor(
            None,
            self.rag_system.retrieve,
            query
        )

    async def generate_async(self, query: str, context: List[dict]) -> str:
        """Async generation"""
        loop = asyncio.get_event_loop()
        return await loop.run_in_executor(
            None,
            self.rag_system.generate_answer,
            query,
            context
        )

    async def query_async(self, query: str) -> dict:
        """Complete async RAG pipeline"""
        # Retrieve and generate can't be parallelized (sequential dependency)
        # But useful for handling multiple queries concurrently
        context = await self.retrieve_async(query)
        answer = await self.generate_async(query, context)

        return {
            'question': query,
            'answer': answer,
            'sources': context
        }
```

```
    async def batch_query(self, queries: List[str]) -> List[dict]:
        """Process multiple queries concurrently"""
        tasks = [self.query_async(q) for q in queries]
        return await asyncio.gather(*tasks)

# Usage
async_rag = AsyncRAG(rag_system)
results = await async_rag.batch_query([
    "What is RAG?",
    "What are embeddings?",
    "How does retrieval work?"
])
```

Monitoring and Logging

```
import logging
import time
from dataclasses import dataclass
from typing import Optional

@dataclass
class RAGMetrics:
    query: str
    retrieval_time: float
    generation_time: float
    total_time: float
    num_chunks_retrieved: int
    answer_length: int
    model_used: str

class MonitoredRAG:
    def __init__(self, rag_system, logger: Optional[logging.Logger] = None):
        self.rag_system = rag_system
        self.logger = logger or logging.getLogger(__name__)
        self.metrics = []

    def query(self, question: str) -> dict:
        """Query with monitoring"""
        start_time = time.time()

        # Retrieval
        retrieval_start = time.time()
        retrieved = self.rag_system.retrieve(question)
        retrieval_time = time.time() - retrieval_start

        # Generation
        generation_start = time.time()
        answer = self.rag_system.generate_answer(question, retrieved)
        generation_time = time.time() - generation_start

        total_time = time.time() - start_time

        # Log metrics
        metrics = RAGMetrics(
```

```
            query=question,
            retrieval_time=retrieval_time,
            generation_time=generation_time,
            total_time=total_time,
            num_chunks_retrieved=len(retrieved),
            answer_length=len(answer),
            model_used=self.rag_system.config.LLM_MODEL
        )

        self.metrics.append(metrics)
        self.logger.info(f"Query processed in {total_time:.2f}s (retrieval: {retrieval_time:.2

        return {
            'question': question,
            'answer': answer,
            'sources': retrieved,
            'metrics': metrics
        }

    def get_average_metrics(self) -> dict:
        """Get average performance metrics"""
        if not self.metrics:
            return {}

        return {
            'avg_total_time': sum(m.total_time for m in self.metrics) / len(self.metrics),
            'avg_retrieval_time': sum(m.retrieval_time for m in self.metrics) / len(self.metri
            'avg_generation_time': sum(m.generation_time for m in self.metrics) / len(self.met
            'total_queries': len(self.metrics)
        }
```

What You've Learned

- ■ Advanced chunking strategies (semantic, recursive, parent-child)
- ■ Hybrid search combining semantic and keyword matching
- ■ Re-ranking to improve retrieval quality
- ■ Query expansion and multi-query retrieval
- ■ Self-querying for automatic filter extraction
- ■ Production optimizations (caching, async, monitoring)

Best Practices Summary

---

1. **Start simple**: Basic RAG first, then add complexity

2. **Measure impact**: A/B test each technique

3. **Chunk wisely**: Experiment with different strategies

4. **Re-rank when possible**: Huge quality improvement

5. **Monitor everything**: Track metrics in production

6. **Cache aggressively**: Many queries are similar

7. **Use hybrid search**: Especially for specific terms

8. **Optimize for your use case**: No one-size-fits-all

Practice Exercises

1. **Implement hybrid search**: Add BM25 to your RAG system

2. **Add re-ranking**: Use cross-encoder to improve results

3. **Try semantic chunking**: Compare with fixed-size chunking

4. **Build monitoring**: Track query performance metrics

5. **Optimize for production**: Add caching and async processing

Further Reading

- Advanced RAG Techniques (blog post)
- RAG From Scratch (videos)
- LangChain Advanced RAG
- Anthropic RAG Cookbook

← Lesson 5: Building a Simple RAG System | Back to README